# Raspberry Pi 5 GPIO Control

## Senior Design

Prof. Nelson

Aerospace Engineering

# Lecture Goals

- Understand Raspberry Pi 5 GPIO architecture
- Control relay outputs with Python
- Control relay outputs with C using libgpiod
- Apply deterministic startup/shutdown safety
- Demos
  - Simple on/off
  - Implement sequential relay activation
  - Implement keyboard-driven relay toggling

# RPi 5 GPIO Architecture

- GPIO handled by RP1 I/O controller
- Legacy RPi.GPIO deprecated
- This means that older examples may not work!
    - Espcially C, older libraries do not work

GPIO devices exposed as:

```
/dev/gpiochip0
```

# RPi 5 firmware

With the new chip, the following are used for talking to the GPIO pins

- Python - *gpiozero*
- C - *libgpiod* (Python uses this too)

These should be installed, but if not...

```
sudo apt update
sudo apt install python3-gpiozero
sudo apt install libgpiod-dev gpiod
```

# Looking at the GPIO pins

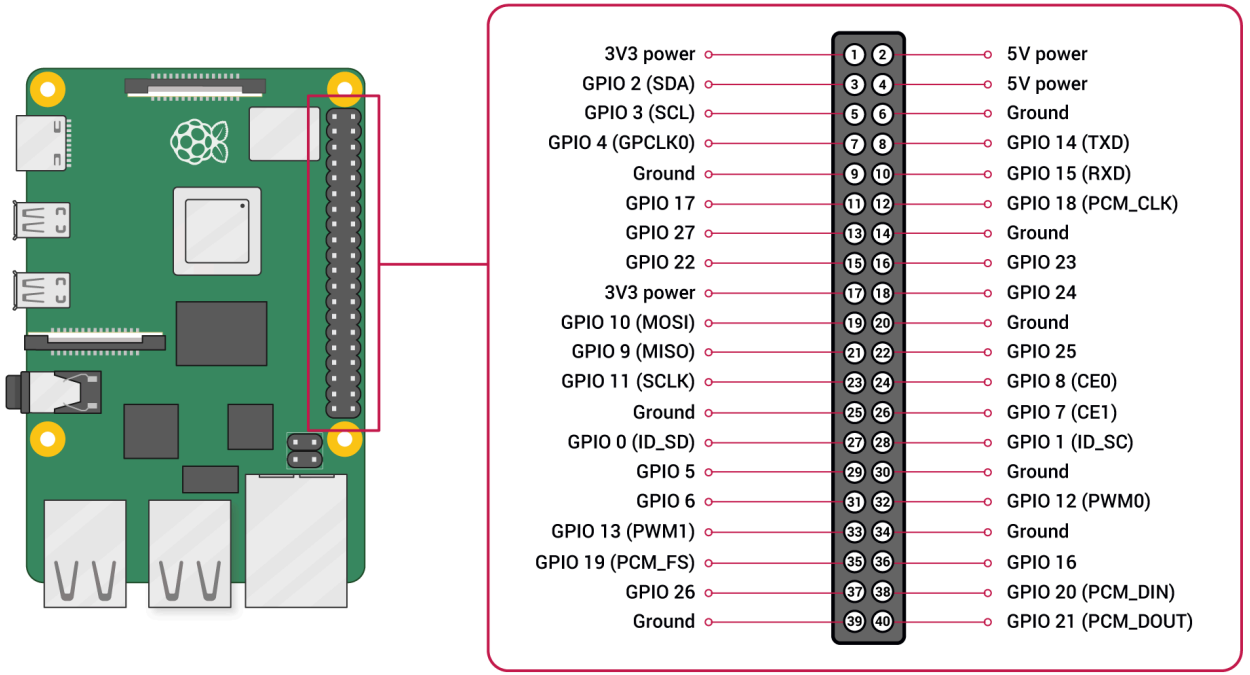
There are a few ways we can look at what pins we have availble.

- Look it up in the Raspberry Pi Documntation page
- Use pinout.xyz websie
- Use the built-in command on the RPi

```
pinout
```

# GPIO Diagram

Note that GPIO names do *not* match pin numers!



# GPIO Hardware

## Voltages

Two 5V pins and two 3V3 pins are present on the board, as well as a number of ground pins (0V), which are unconfigurable. The remaining pins are all general purpose 3V3 pins, meaning outputs are set to 3V3 and inputs are 3V3-tolerant.

# GPIO Hardware

## Outputs

A GPIO pin designated as an output pin can be set to high (3V3) or low (0V).

## Inputs

A GPIO pin designated as an input pin can be read as high (3V3) or low (0V). This is made easier with the use of internal pull-up or pull-down resistors. Pins GPIO2 and GPIO3 have fixed pull-up resistors, but for other pins this can be configured in software.

# Other modes

As well as simple input and output devices, the GPIO pins can be used with a variety of alternative functions, some are available on all pins, others on specific pins.

- PWM (pulse-width modulation)
  - Software PWM available on all pins
  - Hardware PWM available on GPIO12, GPIO13, GPIO18, GPIO19
- SPI
  - SPI0: MOSI (GPIO10); MISO (GPIO9); SCLK (GPIO11); CE0 (GPIO8), CE1 (GPIO7)
  - SPI1: MOSI (GPIO20); MISO (GPIO19); SCLK (GPIO21); CE0 (GPIO18); CE1 (GPIO17); CE2 (GPIO16)

# Other modes

- I2C
  - Data: (GPIO2); Clock (GPIO3)
  - EEPROM Data: (GPIO0); EEPROM Clock (GPIO1)
- Serial
  - TX (GPIO14); RX (GPIO15)

**THERE IS NO ANALOG INPUT ON A RAPSBERRY PI!**

# GPIO Hardware Considerations

- Relay coils must NOT be driven directly (why we use the relay board)
- Each pin on the Raspberry Pi is only 3.3V and anything more can damage it
- Each pin can only handle a very small amount of current (hence why we use these switches)
- Boot state of GPIO pins is undefined!
- Always enforce startup reset

# Demo Hardware Setup

- Raspberry Pi 5
- 24-channel relay module
- GPIO0–GPIO23 used
- Keyboard input via terminal

Demo objectives:

- Sequential relay cycling
- WASD toggle control

# Python GPIO Fundamentals

```
from gpiozero import LED
# Define the switches and the associated GPIO pin
sw1 = LED(0)
```

Note the number refers to the GPIO number, not the pin number!

# Python GPIO Fundamentals

From here, setting the state is easy

```
sw1.off()
sw1.on()
```

Of course this would run really fast, so we can slow it down with

```
sleep(1)
```

# Python GPIO Fundamentals

What if we want to define and call all 24 relays?

# Python GPIO Fundamentals

For Loop!

```
RELAY_PINS = range(24)
DELAY = 1

relays = [LED(pin) for pin in RELAY_PINS]
```

# Python GPIO Fundamentals

And of course, if we wish to iterate through all of them, we use a for loop again.

```
for relay in relays:
        relay.on()
        sleep(DELAY)
        relay.off()
```

# Safety Reset Philosophy

It should be noted that the raspberry pi, may set the state of the pins to random values. To help with this, we should make sure all relays are off.

```
for r in relays:
    r.off()
```

# Python Sequential Relay Demo

```python
from gpiozero import LED
from time import sleep

relays = [LED(pin) for pin in range(24)]

for r in relays:
    r.off()

while True:
    for r in relays:
        r.on()
        sleep(1)
        r.off()
```

# Python WASD Toggle Demo

```
import sys, termios, tty, select
from gpiozero import LED
relays = [LED(pin) for pin in range(24)]
state = [False]*24
key_map = {'w':0,'a':1,'s':2,'d':3}
for r in relays:
    r.off()
old = termios.tcgetattr(sys.stdin)
tty.setcbreak(sys.stdin.fileno())
try:
    while True:
        if select.select([sys.stdin],[],[],0)[0]:
            k = sys.stdin.read(1).lower()
            if k in key_map:
                i = key_map[k]
                state[i] = not state[i]
                relays[i].on() if state[i] else relays[i].off()
finally:
    termios.tcsetattr(sys.stdin, termios.TCSADRAIN, old)
```

# Python Demos

I recommend you do **not** use something like Thonny to run these. In my testing, Thonny seems to have an issue accessing the GPIO pins and then locks them out from other programs, resulting in you having to close Thonny. Instead, run these in the terminal like this:

```
python gpio.py
```

or

```
python3 gpio.py
```

If you get an error about accessing the GPIO pins, try using sudo.

```
sudo python gpio.py
```

# Using C/C++

Yes, of course you can use C, but...

- C programs must be compiled (GCC is fine)
- The Raspberry Pi 5 changed the chip, so older C libraries DO NOT WORK!
- Use libgpiod (should already be installed by default)

# C Define the pins

```c
const char *chip_path = "/dev/gpiochip0";
const unsigned int sw1 = 0;  // GPIO0
// Optional: default output low
    gpiod_line_settings_set_output_value(settings, GPIOD_LINE_VALUE_INACTIVE);

    if (gpiod_line_config_add_line_settings(line_cfg, &sw1, 1, settings) < 0) {
        perror("gpiod_line_config_add_line_settings");
        goto cleanup;
    }
```

# C Sequential Relay Demo (Core Logic)

```c
for (unsigned int i = 0; i < 24; i++) {
    gpiod_line_request_set_value(req, i, ON);
    sleep(1);
    gpiod_line_request_set_value(req, i, OFF);
}
```

# C Keyboard Toggle Demo (Core Logic)

```c
switch (ch) {
    case 'w': idx = 0; break;
    case 'a': idx = 1; break;
    case 's': idx = 2; break;
    case 'd': idx = 3; break;
}
state[idx] = !state[idx];
gpiod_line_request_set_value(req, idx,
    state[idx] ? ON : OFF);
```

# C Compiling

You can use GCC to compile your C code. However, you must link to the libgpiod library. For example:

```
gcc relay_wasd_toggle.c -lgpiod -o relay_wasd_toggle
sudo ./relay_wasd_toggle
```

My testing has not required the use of sudo, but if it doesn't work, then you can use sudo as well. Yes, you can use g++ if you want to do C++.

# Summary

A few things to wrap things up.

- Python should work just fine
- C is going to be more advanced and support on the RPi 5 is not as great
- Yes, other things like XBox controllers or even hooking up buttons to make your own controller are also possible (reverse from output to input on the pins)
- The rest is up to you.
- For all that is good and holy, do everyone a favor and *think the problem through*

Why yes, I do have the examples in a GitHub repo:
https://github.com/matgyver/aere_capstone_space

# Questions